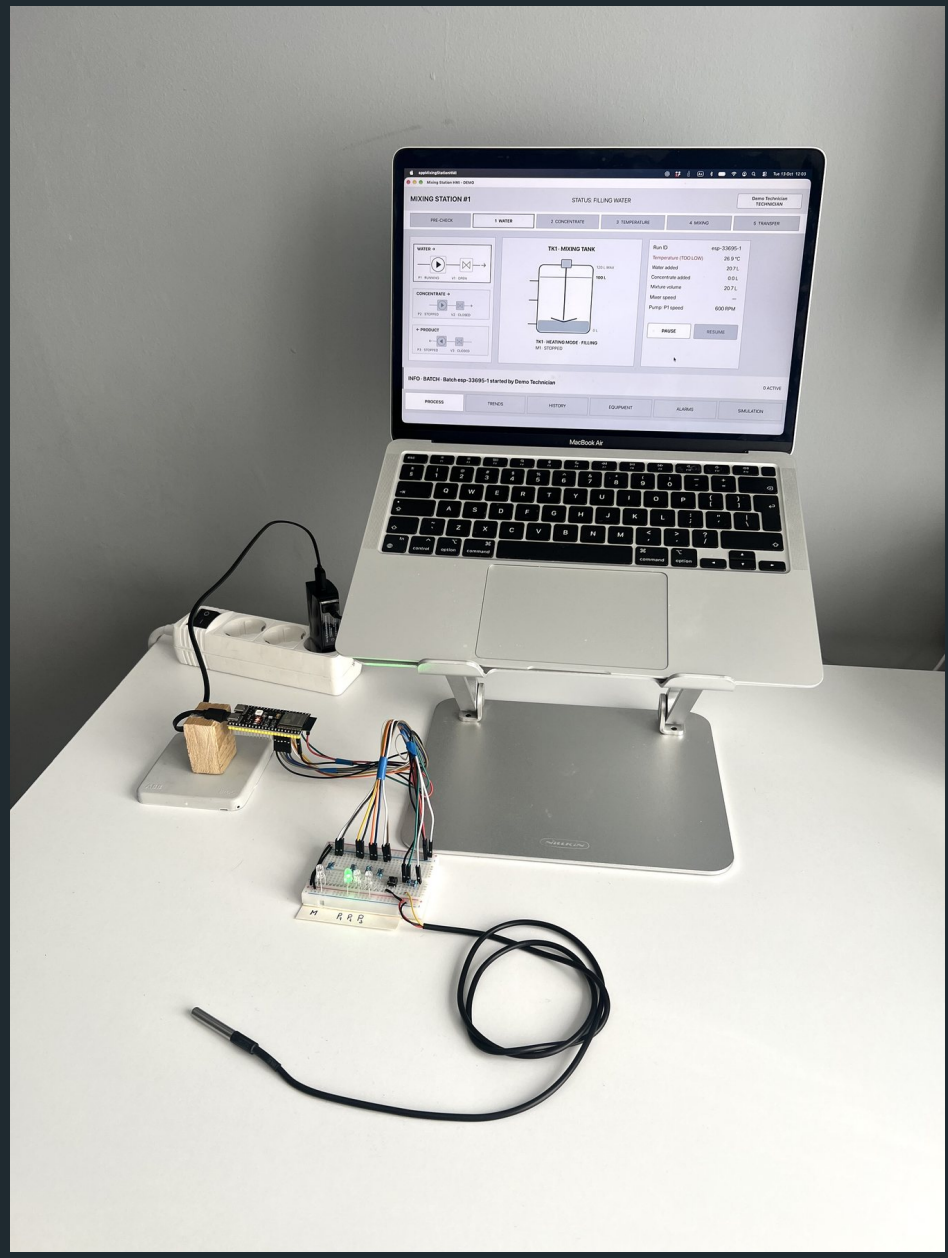
# MixingStationHMI

## Qt HMI + ESP32-S3 controller

Distributed batch-control portfolio prototype

- C++17 / Qt 6 / QML / SQLite
- ESP-IDF 6.1 / FreeRTOS / Modbus TCP
- Wi-Fi / GPIO / DS18B20
- Automated Qt and firmware CI builds

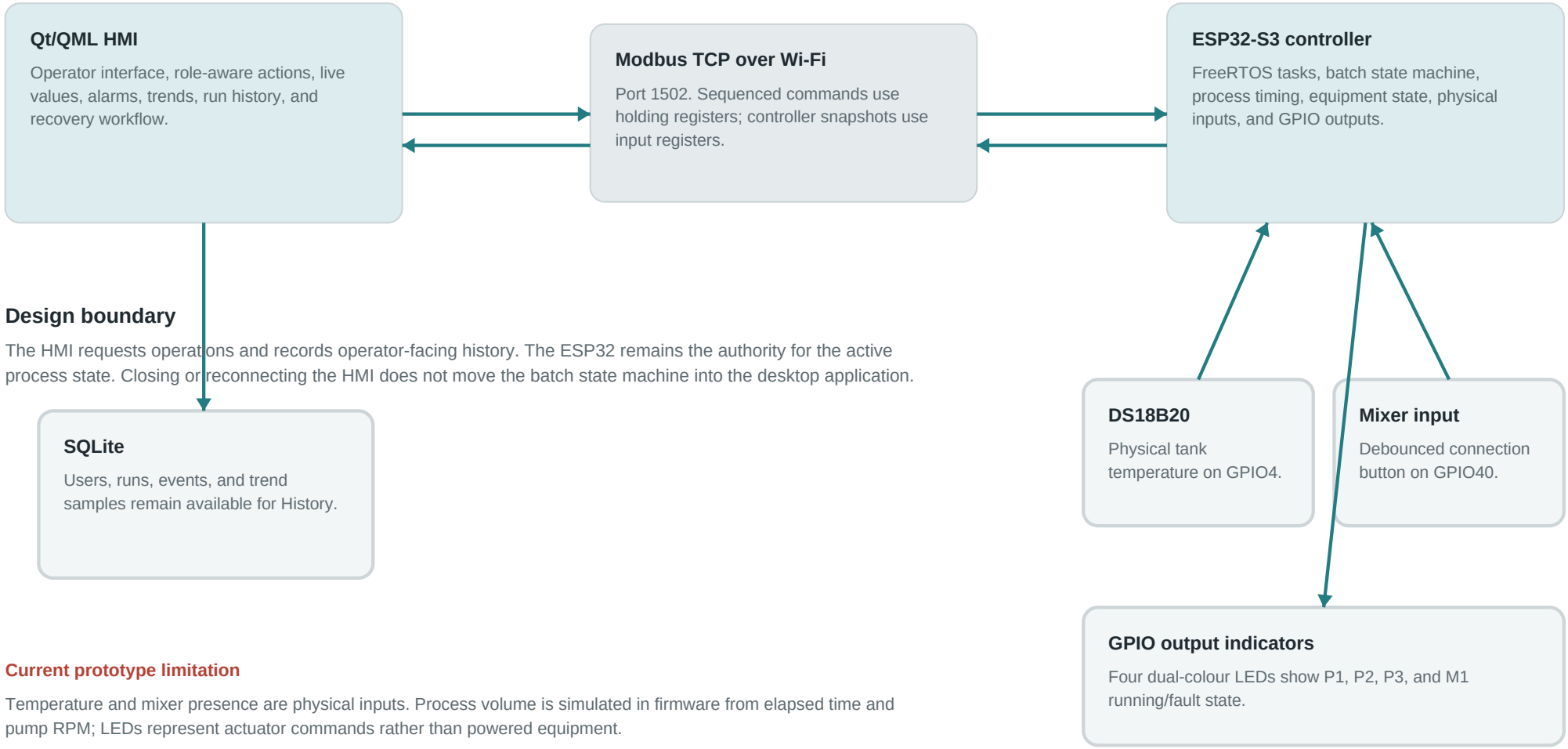



Hardware integration edition | October 2026

https://github.com/LukaszCitko/QtPortfolio/tree/main/projects/MixingStationHMI

# System architecture

The HMI and controller have separate responsibilities and share an explicit Modbus register protocol.




## Design boundary

The HMI requests operations and records operator-facing history. The ESP32 remains the authority for the active process state. Closing or reconnecting the HMI does not move the batch state machine into the desktop application.

## Current prototype limitation

Temperature and mixer presence are physical inputs. Process volume is simulated in firmware from elapsed time and pump RPM; LEDs represent actuator commands rather than powered equipment.

# Batch sequence and controller protocol

A deterministic process sequence is exposed as scaled 16-bit register values.

**PRE-CHECK**
Operator selected, mixer connected, no blocking faults.

→

**70 L WATER**
P1 and V1 active. Volume increases from pump RPM.

→

**30 L CONCENTRATE**
P2 and V2 active; M1 runs at dosing speed.

→

**30-36 C CHECK**
Physical DS18B20 value gates entry to mixing.

→

**8 S MIXING**
M1 runs at 800 RPM; pause stops the timer.

→

**TRANSFER**
Operator starts P3/V3 transfer; controller returns idle.

## Protocol responsibilities

| Register area | Direction | Purpose |
| --- | --- | --- |
| **Holding registers** | HMI -> controller | Command code and sequence, pump target RPM, simulated fault mask |
| **Input registers** | Controller -> HMI | Protocol version, heartbeat, process state, progress, run identity, volumes, temperature, RPM, status and faults |
| **Command result** | Controller -> HMI | Accepted, duplicate sequence, invalid state, or unsupported command |
| **Scaling** | Shared contract | Temperature x10, volume in decilitres, progress in permille, RPM as integer |

**Restart recovery**

A changed controller session ID during an active run marks that run as INTERRUPTED. New batches remain blocked until the operator confirms that the tank has been recovered.

# Electrical schematic

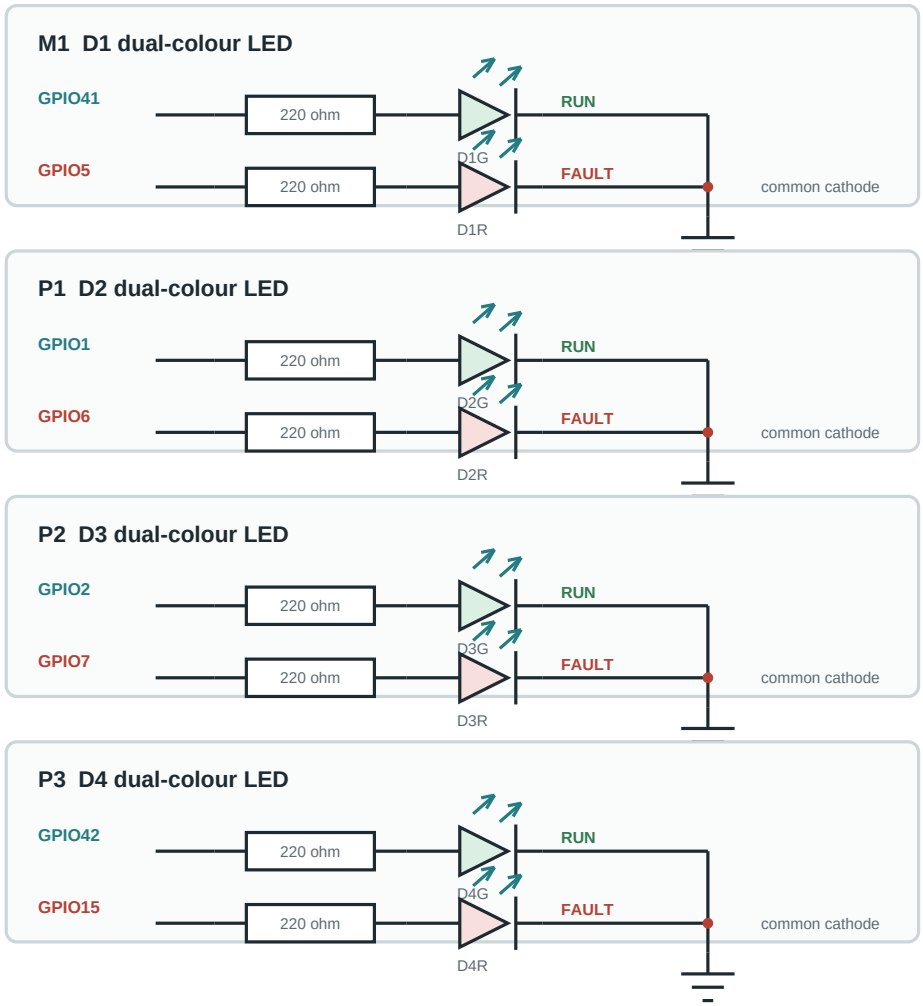
Standard component symbols show the exact 3.3 V prototype connections used by the ESP32-S3 firmware.

## U0 ESP32-S3

DevKitC-1 / 3.3 V logic

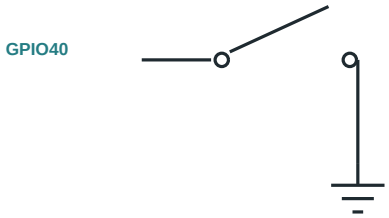
| | |
|---|---|
| GPIO41 | M1 RUN |
| GPIO5 | M1 FAULT |
| GPIO1 | P1 RUN |
| GPIO6 | P1 FAULT |
| GPIO2 | P2 RUN |
| GPIO7 | P2 FAULT |
| GPIO42 | P3 RUN |
| GPIO15 | P3 FAULT |
| GPIO40 | MIXER INPUT |
| GPIO4 | 1-WIRE DATA |

**3V3 / GND**




Four three-lead common-cathode LEDs; each colour channel has its own 220 ohm resistor.

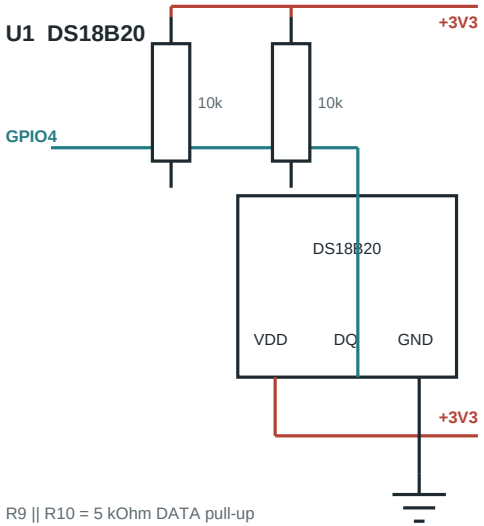
## S1 mixer connection




Normally open / active low

ESP32 internal pull-up enabled

## U1 DS18B20

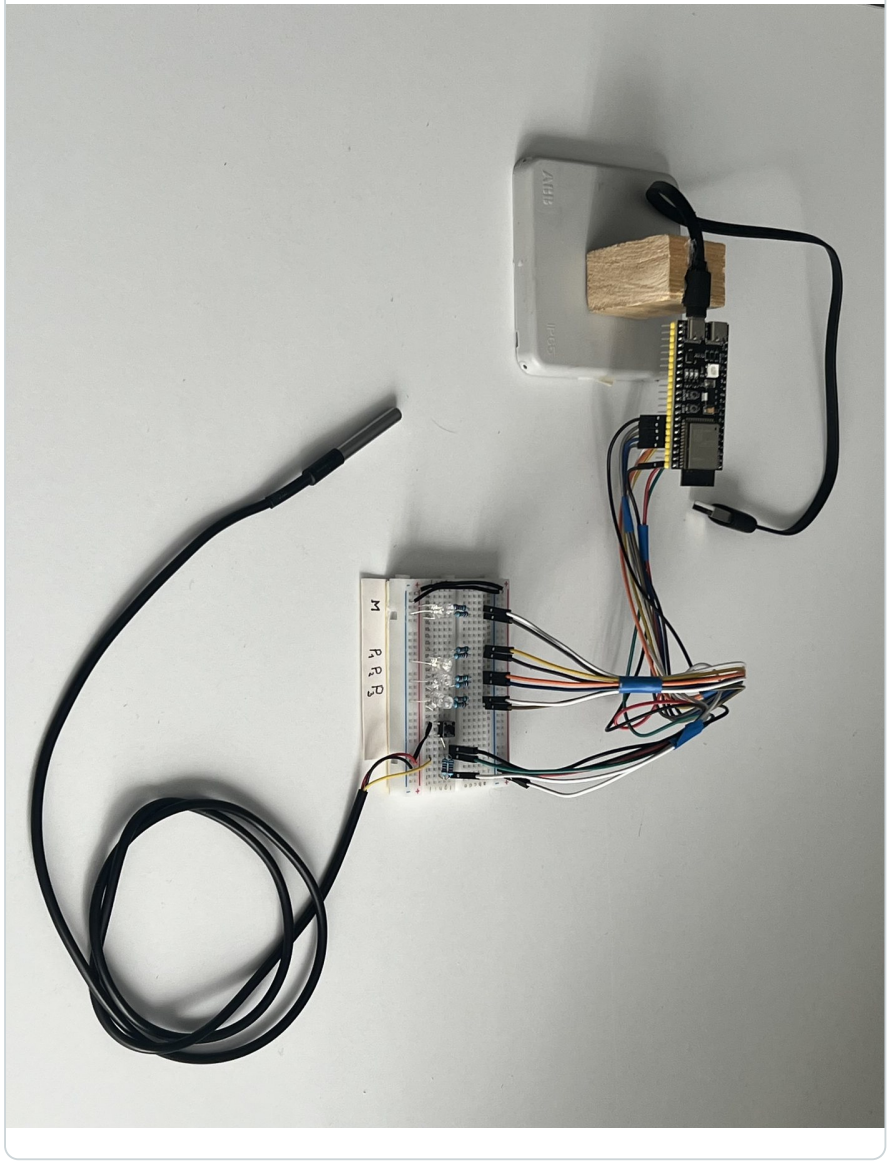


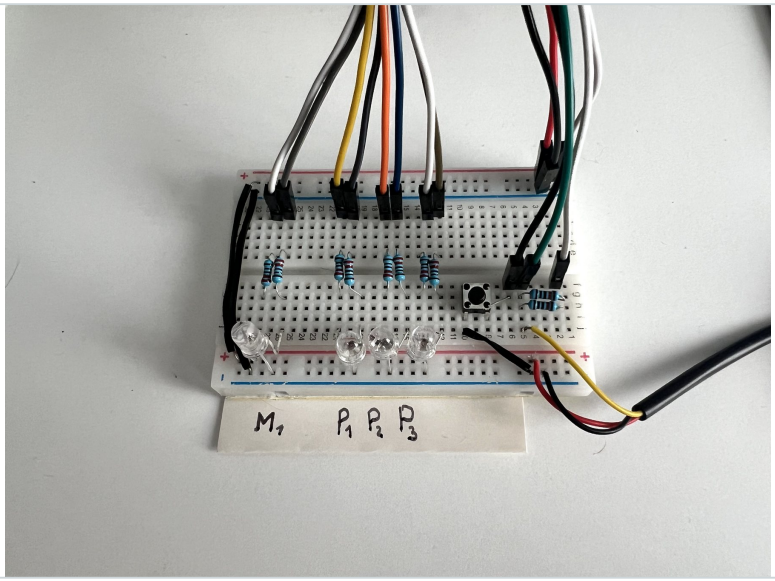

R9 || R10 = 5 kOhm DATA pull-up

GPIO drives indicators only; real actuators require isolated drivers and safety circuits.

# Hardware prototype

ESP32-S3, physical temperature sensing, discrete input, and visible output/fault state.



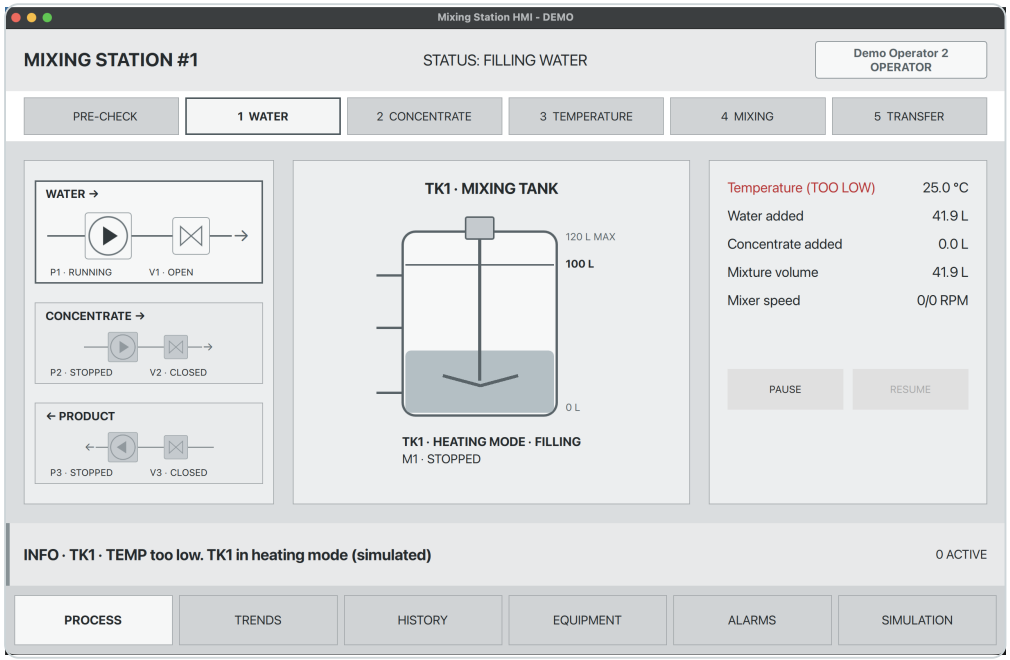

## GPIO map

| | |
|---|---|
| **GPIO1 / GPIO2 / GPIO42** | P1 / P2 / P3 green running channels |
| **GPIO41** | M1 green running channel |
| **GPIO6 / GPIO7 / GPIO15** | P1 / P2 / P3 red fault channels |
| **GPIO5** | M1 red fault channel |
| **GPIO40** | Mixer connection button, internal pull-up |
| **GPIO4** | DS18B20 data, 2 x 10 kOhm parallel pull-up |

ESP32-S3 development board and breadboard test rig. The prototype is powered over USB; status is communicated to the HMI over Wi-Fi.

# Process visibility

The same controller snapshot drives the process screen, equipment cards, alarms, and persisted trends.




### Process view

The HMI identifies the active stage, highlights running devices, shows the controller run ID, presents actual/target RPM, and enables only actions valid in the current process state.



### Physical prototype during water filling

The P1 output indicator follows the controller equipment mask while the Qt application reads volume, temperature, state, heartbeat, and fault registers over Modbus TCP.

### Live process values

Water, concentrate, mixture volume, temperature, mixer speed, and active pump speed.

### Action feedback

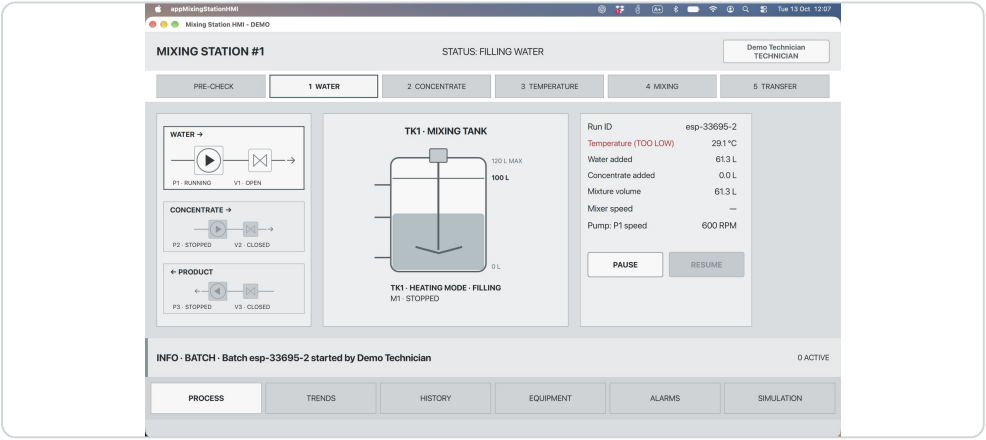
Pause/resume selection, pending command state, transfer permission, and recovery blocking.

### Equipment navigation

Selecting a faulty device or alarm opens the related Equipment card.

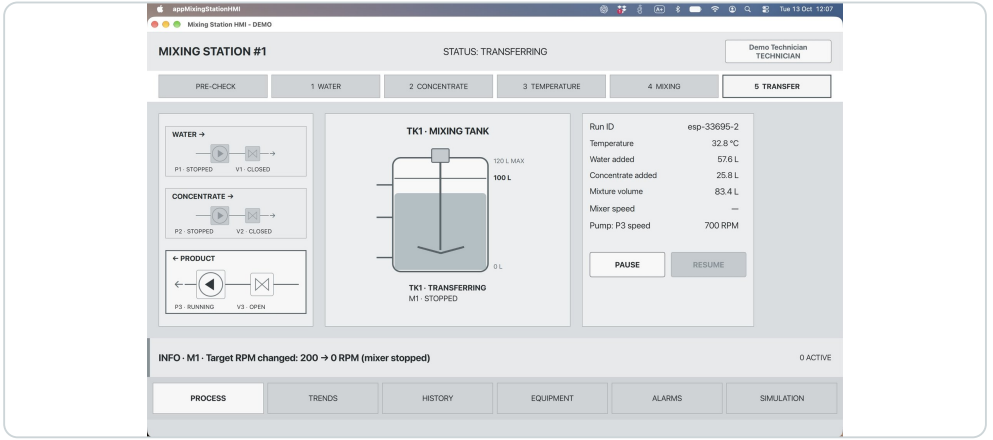
# Desktop HMI in hardware mode

The interface uses the live ESP32-S3 snapshot across process, transfer, trend, and alarm views.



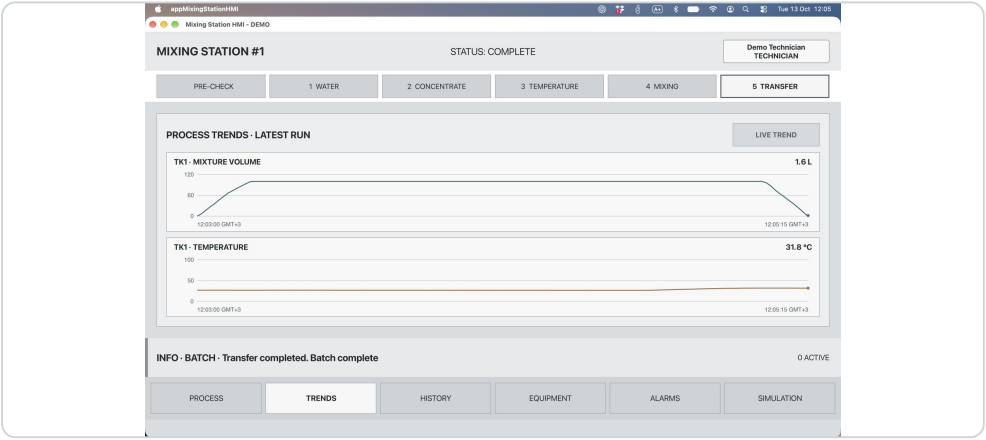

### Water filling

P1 and V1 active; live volume and RPM come from the controller snapshot.



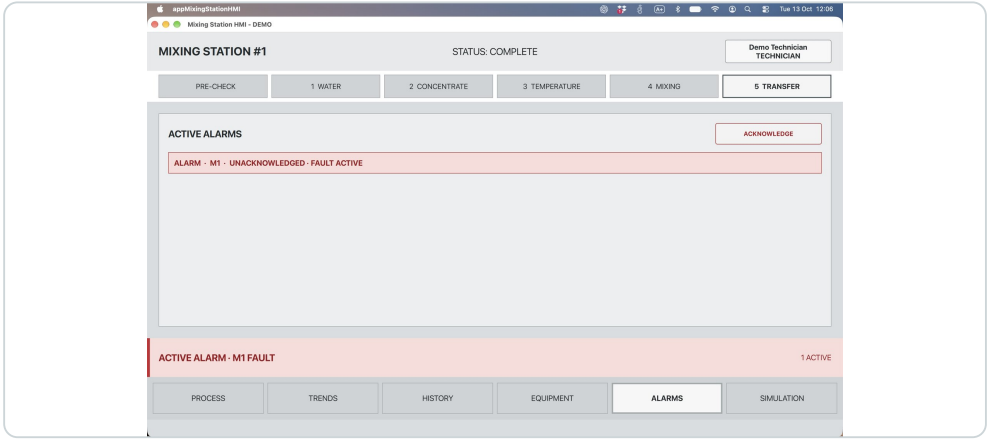

### Transfer

The operator-approved transfer stage activates P3 and reports draining progress.




### Live trends

Temperature, volume, and equipment values remain visible over the batch timeline.

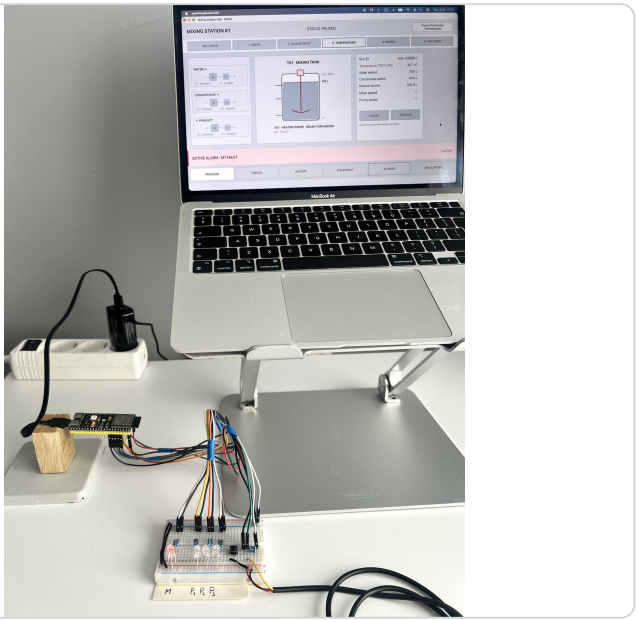



### Alarms

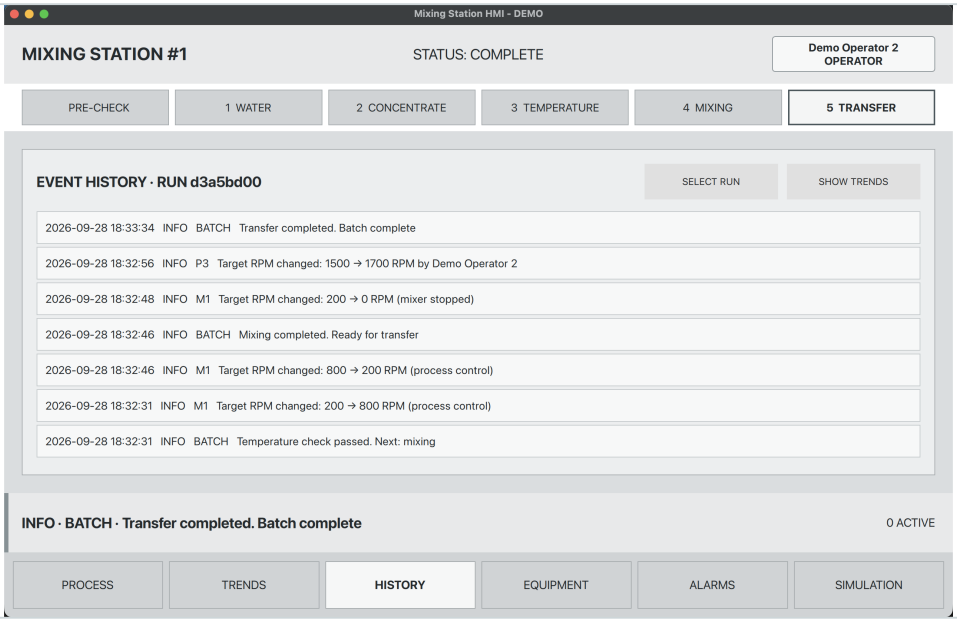
Retained faults, acknowledgement, reset permissions, and navigation share one workflow.

# Faults, recovery, and persisted history

Abnormal states are visible on both the hardware prototype and the operator interface.



**M1 fault: physical red LED and paused HMI**




**Run history and event timeline**

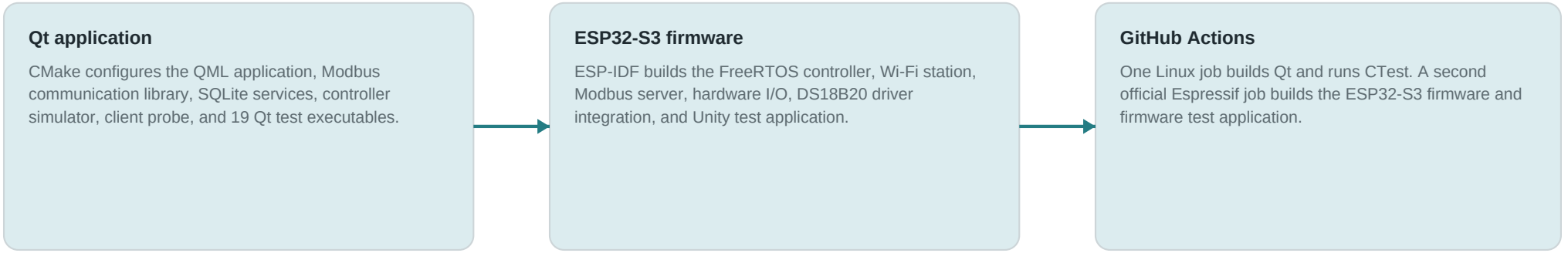
- Fault bits are retained until an authorized reset command is accepted.
- A fault pauses the batch and preserves the last active stage for display.
- Alarm acknowledgement stops attention flashing without clearing the device fault.
- The selected run links events and trend samples through a shared run ID.

### Controller restart workflow

1. HMI detects a changed controller session ID.
2. Active run is completed in SQLite with outcome INTERRUPTED.
3. Start Batch remains blocked because tank state is unknown.
4. Recovery confirmation records an event and clears the block.

# Build, test, and project scope

The repository contains independently buildable Qt and ESP-IDF targets with automated CI checks.




## Validation performed

- Complete demo and hardware-mode batch cycles
- Pause/resume and fault recovery
- Temperature gate and physical DS18B20 changes
- HMI disconnect/reconnect during an active run
- Controller restart detection and interrupted-run persistence
- Qt tests, firmware Unity tests, and clean CI builds

## Portfolio scope

The prototype demonstrates software architecture, embedded communication, test design, and operator-interface behaviour. It is not a production safety controller and does not include powered actuators, certified interlocks, emergency-stop hardware, physical flow/level sensing, or authenticated operator identities.

### Two ways to run

| | |
|---|---|
| **Demo** | Start without arguments; Qt owns the simulated process. |
| **Hardware** | Use --controller-host <IP>; ESP32 owns the process state. |

## Source and detailed build instructions

https://github.com/LukaszCitko/QtPortfolio/tree/main/projects/MixingStationHMI